You said:

from basic to advanced complete dictionary and set in python

ChatGPT said:

Python Dictionary and Set: Complete Guide from Basic to Advanced

This guide covers Python dictionaries (`dict`) and sets (`set`) from beginner concepts to advanced operations, coding interview questions, dry runs, common mistakes, and practical data-science examples.

1. First, understand the difference

Dictionary — `dict`

Stores information as **key-value pairs**.

```
student = {
    "name": "Aamir",
    "age": 22,
    "marks": 85
}
print(student["name"])  # Aamir
```

Real-world example: student records, product prices, employee details.

Set — `set`

Stores **unique elements**, removing duplicates.

```
numbers = {10, 20, 10, 30}
print(numbers)  # {10, 20, 30}
```

Real-world example: unique visitors, unique email addresses, common elements between lists.

2. Dictionary vs Set vs List vs Tuple

Feature	Dictionary	Set	List	Tuple
Syntax	`{"a": 1}`	`{1, 2}`	`[1, 2]`	`(1, 2)`

Stores	Key-value pairs	Unique values	Values	Values
Duplicates	Values yes; keys no	No	Yes	Yes
Ordered	Yes, insertion order	No guaranteed order	Yes	Yes
Mutable	Yes	Yes	Yes	No
Indexing	By key	No	Yes	Yes
Main use	Lookup by key	Uniqueness and membership	Changeable sequence	Fixed sequence

Important: `{}` creates an empty dictionary, not a set. To create an empty set, use `set()`.

Part A — Python Dictionary (`dict`)

3. Dictionary basics

A dictionary maps each unique, hashable key to a value.

```
student = {
    "name": "Aamir",
    "age": 22,
    "course": "AI-ML",
    "marks": 85
}
```

Visualize it like this:

Key	Value
name	"Aamir"
age	22
course	"AI-ML"
marks	85

The key identifies the value. For example, `"marks"` maps to `85`.

Important properties

1. Keys must be unique.
2. Values can be duplicated.
3. Keys must be hashable, such as strings, integers, or tuples containing only hashable elements.
4. Values can be any type, including lists, sets, or other dictionaries.
5. Dictionaries are mutable: you can add, update, or delete entries.
6. Modern Python dictionaries preserve insertion order.

```
data = {
    "a": 10,
    "b": 20,
    "a": 30
}

print(data)
# {'a': 30, 'b': 20}
```

Why? The second `"a"` overwrites the first value. The key still appears in its original position.

4. Creating dictionaries

```
# 1. Literal
d1 = {"name": "Aamir", "age": 22}

# 2. Empty dictionary
d2 = {}

# 3. dict() constructor
d3 = dict(name="Aamir", age=22)

# 4. From key-value pairs
d4 = dict([("a", 1), ("b", 2)])

# 5. From keys with the same initial value
d5 = dict.fromkeys(["a", "b", "c"], 0)

# 6. Dictionary comprehension
squares = {n: n**2 for n in range(1, 5)}

print(d4)      # {'a': 1, 'b': 2}
```

```
print(d5)      # {'a': 0, 'b': 0, 'c': 0}
print(squares) # {1: 1, 2: 4, 3: 9, 4: 16}
```

Important constructor warning

```
# Correct
student = {"name": "Aamir"}

# Incorrect: keys must be hashable
# data = {[1, 2]: "value"}
```

Lists and dictionaries cannot be dictionary keys because they are mutable and unhashable.

5. Accessing dictionary values

A. Square brackets — `d[key]`

```
student = {"name": "Aamir", "marks": 85}

print(student["name"])  # Aamir
print(student["marks"]) # 85

# student["age"]  # KeyError
```

Use square brackets when you expect the key to exist.

B. The `get()` method

Syntax:

```
dictionary.get(key, default=None)
```

- `key` : the key you want to find.
- `default` : optional fallback value if the key is missing.

```
student = {"name": "Aamir", "marks": 85}

print(student.get("name"))          # Aamir
print(student.get("age"))           # None
print(student.get("age", 0))        # 0
```

Interview question: `d[key]` vs `d.get(key)`?

- `d[key]` raises `KeyError` if the key is missing.
- `d.get(key)` returns `None` by default or your specified fallback.

One subtle point: `get()` returns `None` for a missing key by default, but a key that exists with a `None` value also returns `None`.

6. Adding and updating values

```
student = {"name": "Aamir", "marks": 80}

# Add a new key
student["age"] = 22

# Update an existing key
student["marks"] = 90

# Add or update several keys
student.update({"course": "AI-ML", "marks": 95})

print(student)
# {'name': 'Aamir', 'marks': 95, 'age': 22, 'course': 'AI-ML'}
```

`setdefault()` — advanced but useful

Syntax: `dictionary.setdefault(key, default=None)`

- If the key exists, return its current value.
- If it does not exist, insert the key with the default value and return that value.

```
d = {"a": 10}

print(d.setdefault("a", 100))  # 10
print(d.setdefault("b", 20))   # 20
print(d)                       # {'a': 10, 'b': 20}
```

Unlike `get()`, `setdefault()` can modify the dictionary.

7. Deleting dictionary elements

Method	What it does	Result
`pop(key)`	Removes key and returns its value	Value
`pop(key, default)`	Removes key or returns fallback	Value or fallback
`popitem()`	Removes the last inserted item	`(key, value)`
`del d[key]`	Deletes a particular key	No return value
`clear()`	Removes every entry	Empty dictionary

```
d = {"a": 10, "b": 20, "c": 30}

x = d.pop("a")
print(x)  # 10

y = d.pop("z", 0)
print(y)  # 0

item = d.popitem()
print(item)  # ('c', 30)

del d["b"]
print(d)  # {}

d = {"x": 1}
d.clear()
print(d)  # {}
```

Common mistake: `pop()` without a fallback raises `KeyError` if the requested key does not exist.

8. Dictionary methods — complete reference

```
d = {"a": 10, "b": 20, "c": 30}

print(d.keys())              # dict_keys(['a', 'b', 'c'])
print(d.values())            # dict_values([10, 20, 30])
print(d.items())             # dict_items([('a', 10), ...])
print(d.get("a"))            # 10
print(d.copy())              # Shallow copy
```

```
print(d.setdefault("d", 40))  # 40
print(d.pop("d"))             # 40
print(d.popitem())            # ('c', 30)
d.update({"e": 50})
print(d)
d.clear()
print(d)                      # {}
```

Other important dictionary features:

- `dict.fromkeys(iterable, value)` creates a dictionary from keys.
- `len(d)` returns the number of key-value pairs.
- `key in d` checks whether a key exists.
- `d | other` merges dictionaries in Python 3.9+.
- `d |= other` updates a dictionary in place in Python 3.9+.

`keys()`, `values()`, and `items()`

```
student = {"name": "Aamir", "marks": 90}

for key in student.keys():
    print(key)

for value in student.values():
    print(value)

for key, value in student.items():
    print(key, "->", value)
```

The most common interview pattern is:

```
for key, value in student.items():
    print(key, value)
```

Note: `keys()`, `values()`, and `items()` return dynamic dictionary views, not ordinary lists. Changes to the dictionary can be reflected in these views.

9. Dictionary iteration and membership

```
d = {"a": 10, "b": 20, "c": 30}

for key in d:
    print(key)  # a, b, c

print("a" in d)       # True
print(10 in d)        # False
print(10 in d.values())  # True
print(("a", 10) in d.items())  # True
```

Remember:

- `key in d` checks keys.
- `value in d.values()` checks values.
- `pair in d.items()` checks key-value pairs.

10. Dictionary comprehension

A dictionary comprehension is a concise way to create a dictionary.

General syntax:

```
{key_expression: value_expression for item in iterable if condition}
```

Example 1: Squares

```
squares = {n: n**2 for n in range(1, 6)}
print(squares)
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Example 2: Filter values

```
numbers = [1, 2, 3, 4, 5, 6]

result = {n: n**2 for n in numbers if n % 2 == 0}
print(result)
# {2: 4, 4: 16, 6: 36}
```

Example 3: Reverse keys and values

```
original = {"a": 1, "b": 2, "c": 3}

reversed_dict = {value: key for key, value in original.items()}
print(reversed_dict)
# {1: 'a', 2: 'b', 3: 'c'}
```

This reversal works cleanly when values are unique and hashable. If multiple keys share a value, the later entry overwrites the earlier entry.

Example 4: Convert values to uppercase

```
names = {"first": "aamir", "last": "azad"}

result = {key: value.upper() for key, value in names.items()}
print(result)
# {'first': 'AAMIR', 'last': 'AZAD'}
```

11. Nested dictionaries

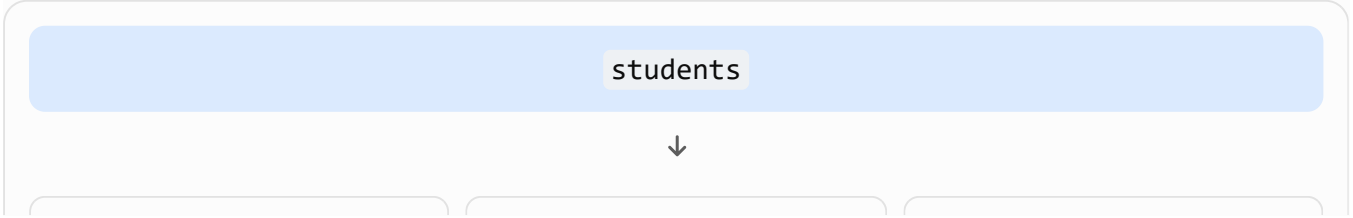
A nested dictionary stores a dictionary inside another dictionary.

```
students = {
    101: {"name": "Aamir", "marks": 85},
    102: {"name": "Rahul", "marks": 92},
    103: {"name": "Sara", "marks": 78}
}

print(students[101]["name"])  # Aamir
print(students[102]["marks"]) # 92

students[103]["marks"] = 88
students[104] = {"name": "Neha", "marks": 95}
```

Visual structure:



ID 101	ID 102	ID 103
Aamir	**Rahul**	**Sara**
Marks: 85	Marks: 92	Marks: 78

Practical use cases include JSON API responses, employee records, configuration files, and grouped datasets.

12. Advanced dictionary concepts

A. Merge two dictionaries

```
a = {"x": 1, "y": 2}
b = {"y": 99, "z": 3}

# Python 3.9+
merged = a | b
print(merged)
# {'x': 1, 'y': 99, 'z': 3}
```

When keys overlap, the right-hand dictionary's values win.

For older Python versions:

```
merged = {**a, **b}
```

Or update an existing dictionary:

```
a.update(b)
```

Unlike `a | b`, `a.update(b)` mutates `a`.

B. Shallow copy vs deep copy

```
original = {"numbers": [1, 2, 3]}

shallow = original.copy()

shallow["numbers"].append(4)
print(original)
# {'numbers': [1, 2, 3, 4]}
```

Why? The outer dictionary was copied, but the nested list is shared.

Use `copy.deepcopy()` when you need to recursively copy nested objects.

```
import copy

original = {"numbers": [1, 2, 3]}
deep = copy.deepcopy(original)

deep["numbers"].append(4)
print(original)  # {'numbers': [1, 2, 3]}
print(deep)      # {'numbers': [1, 2, 3, 4]}
```

C. Mutable default values with `fromkeys()`

Be careful with this pattern:

```
bad = dict.fromkeys(["a", "b"], [])

bad["a"].append(10)
print(bad)
# {'a': [10], 'b': [10]}
```

Both keys refer to the same list.

Safer:

```
good = {key: [] for key in ["a", "b"]}
good["a"].append(10)

print(good)
# {'a': [10], 'b': []}
```

D. Dictionary keys and hashability

Dictionary keys must be hashable. Most immutable built-in values are hashable, but immutability alone is not enough.

```
data = {
    "name": "Aamir",
```

```
    10: "ten",
    (1, 2): "tuple key"
}

# Invalid:
# data[[1, 2]] = "list key"
# TypeError: unhashable type: 'list'
```

A tuple can be a key only if all its elements are hashable:

```
valid = {(1, 2): "valid"}

# Invalid:
# invalid = {(1, [2]): "invalid"}
```

E. `defaultdict` for automatic defaults

```
from collections import defaultdict

counts = defaultdict(int)

for letter in "banana":
    counts[letter] += 1

print(dict(counts))
# {'b': 1, 'a': 3, 'n': 2}
```

`int()` returns `0`, so a missing key automatically begins at zero. This is especially useful for counting and grouping.

F. `Counter` for counting

```
from collections import Counter

words = ["apple", "banana", "apple", "orange", "banana", "apple"]

counts = Counter(words)
print(counts)
# Counter({'apple': 3, 'banana': 2, 'orange': 1})
```

```
print(counts.most_common(2))
# [('apple', 3), ('banana', 2)]
```

`Counter` is a specialized dictionary-like tool for frequency analysis.

G. Dictionary time complexity

Operation	Average time
Access by key	$O(1)$
Insert or update	$O(1)$
Delete by key	$O(1)$
Search for a key	$O(1)$
Search for a value	$O(n)$
Iterate all entries	$O(n)$

These are average-case expectations; worst-case behavior can differ.

Part B — Python Set (`set`)

13. Set basics

A set is a mutable collection of unique, hashable elements.

```
numbers = {10, 20, 10, 30, 20}
print(numbers)
# {10, 20, 30}  (display order may vary)
```

Key properties:

- Duplicate elements are removed.
- Elements must be hashable.
- Sets are mutable.
- Sets do not support indexing or slicing.
- Membership checks are usually $O(1)$ on average.
- Sets do not guarantee a particular iteration order.

Creating sets

```
s1 = {1, 2, 3}
s2 = set([1, 2, 2, 3])
s3 = set("hello")
empty = set()

print(s2)     # {1, 2, 3}
print(empty)  # set()
```

Don't use `{}` for an empty set because it creates a dictionary.

14. Adding elements

`add()` — add one element

```
s = {10, 20}

s.add(30)
s.add(20)  # Duplicate; no change

print(s)   # {10, 20, 30}
```

`update()` — add multiple elements

```
s = {1, 2}

s.update([3, 4, 5])
print(s)  # {1, 2, 3, 4, 5}

s.update({6, 7}, [8, 9])
print(s)
```

`update()` accepts one or more iterables.

15. Removing set elements

Method	Behavior
`remove(x)`	Removes `x`; raises `KeyError` if absent

`discard(x)`	Removes `x` if present; otherwise does nothing
`pop()`	Removes and returns an arbitrary element
`clear()`	Removes all elements

```
s = {10, 20, 30}

s.remove(10)
s.discard(99)  # Safe if missing

item = s.pop() # Which element? Do not assume
print(item)

s.clear()
print(s)       # set()
```

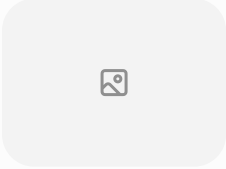
Interview tip: Prefer `discard()` when the element may not exist.

16. Set operations — most important interview topic

Let:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

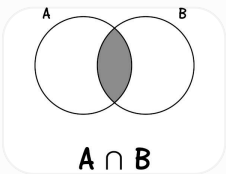
Visual set operations



1. Union — `A | B`

Combines all unique elements from both sets.

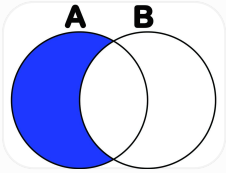
```
A | B
# {1, 2, 3, 4, 5, 6}
```



2. Intersection — `A & B`

Returns elements present in both sets.

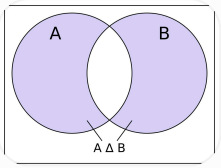
```
A & B
# {3, 4}
```



3. Difference — `A - B`

Returns elements in A but not B.

```
A - B
# {1, 2}
```



4. Symmetric difference — `A ^ B`

Returns elements in either set, but not both.

```
A ^ B
# {1, 2, 5, 6}
```

The same operations can be written using methods:

```
A.union(B)
A.intersection(B)
A.difference(B)
A.symmetric_difference(B)
```

Dry run: intersection

Element	In A?	In B?	Included?
1	Yes	No	No
2	Yes	No	No
3	Yes	Yes	Yes
4	Yes	Yes	Yes
5	No	Yes	No
6	No	Yes	No

Final answer: `{3, 4}`.

17. Updating sets in place

```
A = {1, 2, 3}
B = {3, 4, 5}

A |= B
print(A)  # {1, 2, 3, 4, 5}
```

Operator	In-place equivalent
`A | B`	`A.update(B)`
`A & B`	`A.intersection_update(B)`
`A - B`	`A.difference_update(B)`
`A ^ B`	`A.symmetric_difference_update(B)`

In-place methods modify the original set rather than creating a new set.

18. Subsets, supersets and disjoint sets

```
A = {1, 2}
B = {1, 2, 3, 4}
C = {7, 8}

print(A.issubset(B))       # True
print(B.issuperset(A))     # True
print(A.isdisjoint(C))     # True
print(A.issubset(C))       # False
```

Operators:

```
A <= B  # A is a subset of B
B >= A  # B is a superset of A
A < B   # Proper subset: subset but not equal
B > A   # Proper superset
```

A proper subset means the sets are not equal. A set is disjoint from another if they have no common elements.

19. Membership and comparisons

```
s = {10, 20, 30}

print(20 in s)      # True
print(99 not in s)  # True
print(len(s))       # 3

print({1, 2} == {2, 1})  # True
print({1, 2} == {1, 3})  # False
```

Set equality ignores element order.

20. Set comprehension

Syntax:

```
{expression for item in iterable if condition}
```

Example 1: Squares

```
squares = {n**2 for n in range(1, 6)}
print(squares)
# {1, 4, 9, 16, 25}
```

Example 2: Unique vowels

```
text = "hello world"

vowels = {ch for ch in text if ch in "aeiou"}
print(vowels)
# {'e', 'o'}
```

Example 3: Even numbers

```
evens = {n for n in range(1, 11) if n % 2 == 0}
print(evens)
# {2, 4, 6, 8, 10}
```

Unlike dictionary comprehensions, set comprehensions contain one expression rather than a `key: value` pair.

21. Advanced set concepts

A. `frozenset` — immutable set

A `frozenset` cannot be changed after creation. It supports set operations and can be used as a dictionary key or as an element of another set.

```
fs = frozenset([1, 2, 2, 3])

print(fs)  # frozenset({1, 2, 3})

# fs.add(4)  # AttributeError
# fs.remove(1)  # AttributeError

data = {fs: "numbers"}
print(data[fs])  # numbers

outer = {frozenset([1, 2]), frozenset([2, 3])}
```

Use a regular `set` when you need to add or remove elements. Use `frozenset` when you need an immutable set.

B. Removing duplicates from a list

```
numbers = [4, 2, 4, 1, 2, 3]

unique = list(set(numbers))
print(unique)
```

This removes duplicates, but **does not preserve the original order**.

To preserve order:

```
numbers = [4, 2, 4, 1, 2, 3]

unique = list(dict.fromkeys(numbers))
print(unique)  # [4, 2, 1, 3]
```

This second approach works for hashable elements.

C. Why sets are fast

```
numbers = list(range(100_000))
number_set = set(numbers)

print(99_999 in numbers)     # Linear scan in the worst case
print(99_999 in number_set)  # Average constant-time lookup
```

Operation	Average complexity
`x in s`	$O(1)$
`s.add(x)`	$O(1)$
`s.remove(x)`	$O(1)$
`s.union(t)`	$O(n + m)$
`s.intersection(t)`	Depends on set sizes and implementation
Iterating through a set	$O(n)$

Here, n and m are the sizes of the relevant sets. These are average-case expectations.

D. Why can't a set contain a list?

```
# Invalid
# s = {[1, 2], [3, 4]}
# TypeError: unhashable type: 'list'

# Valid
s = {(1, 2), (3, 4)}
```

A tuple is allowed only if all its elements are hashable.

E. Set operations with multiple sets

```
A = {1, 2, 3, 4}
B = {3, 4, 5}
C = {4, 5, 6}
```

```
print(A.union(B, C))
# {1, 2, 3, 4, 5, 6}

print(A.intersection(B, C))
# {4}
```

Useful when comparing multiple datasets, lists of users, or collections of product IDs.

Part C — Dictionary and Set Coding Interview Questions

Below are common interview problems with solutions and dry runs. Try predicting each output before running the code.

22. Dictionary coding questions

Q1. Count the frequency of each element

```
numbers = [1, 2, 2, 3, 1, 2]

freq = {}

for num in numbers:
    freq[num] = freq.get(num, 0) + 1

print(freq)
# {1: 2, 2: 3, 3: 1}
```

Dry run:

Number	Previous count	New count
1	0	1
2	0	1
2	1	2
3	0	1
1	1	2
2	2	3

Logic: `get(num, 0)` returns the existing count or zero when the key is new.

Q2. Count character frequencies in a string

```
text = "banana"
freq = {}

for ch in text:
    freq[ch] = freq.get(ch, 0) + 1

print(freq)
# {'b': 1, 'a': 3, 'n': 2}
```

Interview extension: ignore spaces, convert letters to lowercase, or find the most frequent character.

Q3. Find the first non-repeating character

```
text = "aabbcdde"

freq = {}

for ch in text:
    freq[ch] = freq.get(ch, 0) + 1

for ch in text:
    if freq[ch] == 1:
        print(ch)
        break

# c
```

Why two loops? The first counts all characters. The second finds the first character with a count of one in the original order.

Q4. Find two numbers that add up to a target

```
numbers = [2, 7, 11, 15]
target = 9

seen = {}
```

```
for i, num in enumerate(numbers):
    need = target - num

    if need in seen:
        print(seen[need], i)
        break

    seen[num] = i

# 0 1
```

Dry run:

Index	Number	Needed	Already seen?
0	2	7	No; store `{2: 0}`
1	7	2	Yes; return indices `0, 1`

Time complexity: $O(n)$ average, compared with $O(n^2)$ for checking every pair.

Q5. Merge two dictionaries and add common-key values

```
a = {"apple": 10, "banana": 20}
b = {"banana": 5, "orange": 15}

result = a.copy()

for key, value in b.items():
    result[key] = result.get(key, 0) + value

print(result)
# {'apple': 10, 'banana': 25, 'orange': 15}
```

Notice that this is different from ordinary dictionary merging: common-key values are added rather than overwritten.

Q6. Find the key with the maximum value

```
marks = {"Aamir": 85, "Rahul": 92, "Sara": 78}
```

```
topper = max(marks, key=marks.get)

print(topper)       # Rahul
print(marks[topper]) # 92
```

The `key=marks.get` argument tells `max()` to compare dictionary values rather than dictionary keys.

Q7. Group words by their first letter

```
words = ["apple", "ant", "banana", "ball", "cat"]

groups = {}

for word in words:
    first = word[0]
    groups.setdefault(first, []).append(word)

print(groups)
# {'a': ['apple', 'ant'],
#  'b': ['banana', 'ball'],
#  'c': ['cat']}
```

This is a common grouping pattern used in data processing.

Q8. Invert a dictionary

```
data = {"a": 1, "b": 2, "c": 3}

result = {value: key for key, value in data.items()}

print(result)
# {1: 'a', 2: 'b', 3: 'c'}
```

Remember that duplicate values cause overwriting during inversion.

Q9. Find common keys between two dictionaries

```
a = {"x": 1, "y": 2, "z": 3}
b = {"y": 20, "z": 30, "w": 40}
```

```
common = a.keys() & b.keys()

print(common)
# {'y', 'z'}
```

Q10. Sort a dictionary by value

```
scores = {"Aamir": 85, "Rahul": 92, "Sara": 78}

result = dict(
    sorted(scores.items(), key=lambda item: item[1])
)

print(result)
# {'Sara': 78, 'Aamir': 85, 'Rahul': 92}
```

For descending order, add `reverse=True` to `sorted()`.

Q11. Remove dictionary entries with values below 50

```
marks = {"A": 45, "B": 80, "C": 30, "D": 90}

result = {
    name: score
    for name, score in marks.items()
    if score >= 50
}

print(result)
# {'B': 80, 'D': 90}
```

Use a new dictionary comprehension rather than deleting entries while iterating over the same dictionary.

Q12. Find the second-highest distinct value

```
marks = {"A": 90, "B": 80, "C": 90, "D": 70}

unique_scores = set(marks.values())

if len(unique_scores) >= 2:
```

```
    second_highest = sorted(unique_scores)[-2]
    print(second_highest)  # 80
else:
    print("No second-highest distinct value")
```

This finds the second-highest **distinct** value, not the second entry after sorting.

23. Set coding questions

Q1. Remove duplicates from a list

```
numbers = [1, 2, 2, 3, 4, 4, 5]

result = list(set(numbers))
print(result)  # Order is not guaranteed
```

Order-preserving alternative:

```
result = list(dict.fromkeys(numbers))
print(result)  # [1, 2, 3, 4, 5]
```

Q2. Find common elements between two lists

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]

common = set(a) & set(b)
print(common)  # {3, 4}
```

Q3. Find elements that occur in only one list

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]

result = set(a) ^ set(b)
print(result)  # {1, 2, 5, 6}
```

This finds the symmetric difference of the two collections.

Q4. Check whether two lists contain the same unique elements

```
a = [1, 2, 2, 3]
b = [3, 2, 1]

print(set(a) == set(b))  # True
```

Important: this ignores duplicates. If duplicate counts matter, compare `Counter(a) == Counter(b)` instead.

Q5. Check whether one list is a subset of another

```
a = [2, 3]
b = [1, 2, 3, 4, 5]

print(set(a).issubset(set(b)))  # True
```

Q6. Find missing numbers from a range

```
numbers = [1, 2, 4, 6, 7]
expected = set(range(1, 8))

missing = expected - set(numbers)
print(missing)  # {3, 5}
```

Q7. Find unique vowels in a string

```
text = "programming interview"

vowels = set(text.lower()) & set("aeiou")
print(vowels)  # {'a', 'e', 'i', 'o'}
```

The output order may vary. Only vowels present in the string are included.

Q8. Find the union of multiple lists

```
lists = [[1, 2], [2, 3], [3, 4]]
```

```
result = set().union(*lists)
print(result)  # {1, 2, 3, 4}
```

The `*lists` syntax unpacks the list of lists into separate arguments.

Q9. Check whether two strings are anagrams

```
a = "listen"
b = "silent"

print(len(a) == len(b) and set(a) == set(b))
# True
```

Caution: this is not a correct general anagram test because sets discard character frequencies. For example, `"aab"` and `"abb"` have the same set of characters but are not anagrams.

Correct solution:

```
from collections import Counter

def are_anagrams(a, b):
    return Counter(a) == Counter(b)

print(are_anagrams("listen", "silent"))  # True
print(are_anagrams("aab", "abb"))       # False
```

Q10. Find the longest consecutive sequence

```
def longest_consecutive(nums):
    values = set(nums)
    longest = 0

    for num in values:
        if num - 1 not in values:
            length = 1

            while num + length in values:
                length += 1

            longest = max(longest, length)
```

```
    return longest

print(longest_consecutive([100, 4, 200, 1, 3, 2]))
# 4
```

Dry run: the consecutive sequence is `1, 2, 3, 4`, so the answer is `4`.

Average time complexity: $O(n)$. The set makes membership checks efficient, and the algorithm only starts a sequence at its smallest element.

Part D — Dictionary vs Set: Interview Questions and Answers

24. Most asked theory questions

1. What is a dictionary in Python?

2. What is a set in Python?

3. Can a dictionary have duplicate keys?

4. Can a dictionary have duplicate values?

5. Can a set contain duplicate elements?

6. Why can't a list be a dictionary key?

7. What is the difference between get() and setdefault()?

8. What is the difference between remove() and discard()?

9. What is a frozenset?

10. What is dictionary comprehension?

11. What is set comprehension?

12. Does a set preserve insertion order?

13. What does popitem() do in a dictionary?

14. What is the average lookup complexity of a dict or set?

15. How do you count frequencies efficiently?

16. How do you find common elements?

Reveal all answers

25. Common mistakes to avoid

Mistake	Correct approach
`d["missing"]` when a key may not exist	Use `d.get("missing")` or check membership
`empty = {}` for an empty set	Use `empty = set()`
Assuming sets are sorted	Use `sorted(my_set)`
Using a list as a set element	Use a hashable representation such as a tuple
Assuming `set(list)` preserves order	Use `dict.fromkeys(list)` if elements are hashable
Using sets for frequency counting	Use `Counter` or a dictionary
Removing dictionary entries while iterating over it	Build a new dictionary or iterate over a copy

Assuming `copy()` recursively copies nested objects	Use `copy.deepcopy()` when appropriate
Comparing anagrams using sets alone	Compare character counts with `Counter`
Assuming `d.get(key, 0)` changes the dictionary	It does not insert the key; use assignment or `setdefault()`

26. Practice exercises

Solve these without looking at the earlier solutions. Select an exercise to check its expected result.

Exercise 1 of 8 Practice mode

Find the frequency of each number

```
nums = [2, 2, 3, 4, 3, 2]
```

Hint: Use a dictionary or Counter.

Show answer Next exercise →

27. Final revision checklist

Your progress 0/11

- ○ Create, access, add, update and delete dictionary entries
- ○ Use get(), setdefault(), update(), pop() and items()
- ○ Write dictionary comprehensions and nested dictionary operations
- ○ Understand hashing, hashable keys, shallow copy and deep copy
- ○ Use Counter and defaultdict for frequency and grouping
- ○ Create sets and use add(), update(), remove() and discard()
- ○ Master union, intersection, difference and symmetric difference
- ○ Understand subsets, supersets, disjoint sets and frozenset
- ○ Remove duplicates and solve common-element problems

- [ ] Solve frequency counting, Two Sum and longest consecutive sequence
- [ ] Explain average time complexity and common interview pitfalls

Reset checklist

Recommended learning order: dictionary basics → dictionary methods → comprehensions → frequency counting → sets and set operations → `Counter` and `defaultdict` → advanced interview problems.

For data science, pay special attention to dictionary-based frequency counting, grouping records, removing duplicates, set intersections, and comparing unique values across datasets. These patterns are useful in Python, Pandas preprocessing, exploratory data analysis, and coding interviews.

You said:

Create, access, add, update and delete dictionary entries
[ ] Use get(), setdefault(), update(), pop() and items()
[ ] Write dictionary comprehensions and nested dictionary operations
[ ] Understand hashing, hashable keys, shallow copy and deep copy
[ ] Use Counter and defaultdict for frequency and grouping
[ ] Create sets and use add(), update(), remove() and discard()
[ ] Master union, intersection, difference and symmetric difference
[ ] Understand subsets, supersets, disjoint sets and frozenset
[ ] Remove duplicates and solve common-element problems
[ ] Solve frequency counting, Two Sum and longest consecutive sequence
[ ] Explain average time complexity and common interview pitfalls
Reset checklist

ChatGPT said:

Great — this is your **Dictionary and Set revision checklist**. Let's turn it into an actionable study tracker. Check each topic as you master it, and use **Reset checklist** to start again whenever you want.

Study progress 0/11

- [ ] **Dictionary CRUD operations**
  Create, access, add, update and delete entries.

- [ ] **Dictionary methods**
  get(), setdefault(), update(), pop() and items().
- [ ] **Comprehensions and nesting**
  Dictionary comprehensions and nested dictionary operations.
- [ ] **Hashing and copying**
  Hashable keys, shallow copy and deep copy.
- [ ] **Counting and grouping**
  collections.Counter and defaultdict.
- [ ] **Set basics**
  Create sets; use add(), update(), remove() and discard().
- [ ] **Set operations**
  Union, intersection, difference and symmetric difference.
- [ ] **Set relationships**
  Subsets, supersets, disjoint sets and frozenset.
- [ ] **Duplicates and common elements**
  Remove duplicates and compare collections.
- [ ] **Interview coding problems**
  Frequency counting, Two Sum and longest consecutive sequence.
- [ ] **Complexity and pitfalls**
  Average time complexity and common interview mistakes.

Start learning | Reset checklist

Recommended approach: For every topic, learn the definition, understand the syntax, run the code, trace it manually, and then solve two or three interview questions without looking at the solution.

You said:

Teach me Python dictionary and set topic 1: Create, access, add, update and delete dictionary entries. Explain step by step with code examples, dry runs, practice questions and interview questions.

ChatGPT said:

Python Dictionary — Topic 1: Create, Access, Add, Update and Delete

We'll learn this like an interview study class: simple definitions, syntax, code examples, dry runs, common mistakes, and practice problems.

1. What is a dictionary?

A **dictionary (`dict`)** is a Python data structure that stores data in **key-value pairs**.

Think of a student record:

Key	Value
`{pair[0]}`	`{pair[1]}`
`{pair[0]}`	`{pair[1]}`
`{pair[0]}`	`{pair[1]}`

- **Key:** The label used to find data, such as `"name"`.
- **Value:** The information associated with the key, such as `"Aamir"`.

Create your first dictionary

```
student = {
    "name": "Aamir",
    "age": 22,
    "marks": 85
}

print(student)
```

Output:

```
{'name': 'Aamir', 'age': 22, 'marks': 85}
```

Remember these rules:

- Keys must be unique.
- Values can be duplicated.
- Keys must be hashable, such as strings, integers, or suitable tuples.
- Dictionaries are mutable, meaning you can change their contents.
- Dictionaries preserve insertion order in modern Python.

2. Create a dictionary

There are several ways to create dictionaries.

```
# Method 1: Curly braces
student = {"name": "Aamir", "age": 22}

# Method 2: Empty dictionary
data = {}

# Method 3: dict() constructor
person = dict(name="Aamir", age=22)

# Method 4: From key-value pairs
marks = dict([("Python", 90), ("Java", 80)])

print(student)
print(data)
print(person)
print(marks)
```

Output:

```
{'name': 'Aamir', 'age': 22}
{}
{'name': 'Aamir', 'age': 22}
{'Python': 90, 'Java': 80}
```

Interview question: How do you create an empty dictionary?

```
d = {}
# Or
d = dict()
```

Both are correct.

3. Access dictionary values

There are two main ways to access a value.

Method A: Square brackets `[]`

```
student = {"name": "Aamir", "age": 22, "marks": 85}

print(student["name"])
print(student["marks"])
```

Output:

```
Aamir
85
```

Dry run:

Step	Expression	Result
1	`student["name"]`	`"Aamir"`
2	`student["age"]`	`22`
3	`student["marks"]`	`85`

What happens if a key does not exist?

```
print(student["city"])
```

This raises a `KeyError` because `"city"` is not a key in the dictionary.

Method B: `get()` method

Use `get()` when a key might be missing.

```
student = {"name": "Aamir", "age": 22}

print(student.get("name"))
```

```
print(student.get("city"))
print(student.get("city", "Unknown"))
```

Output:

```
Aamir
None
Unknown
```

Syntax:

```
dictionary.get(key, default)
```

- `key` : the key to search for.
- `default` : the value to return if the key is missing. If omitted, it defaults to `None` .

Remember: `student["city"]` raises an error, while `student.get("city")` returns `None` .

4. Add new entries

To add a new key-value pair, use square brackets.

```
student = {
    "name": "Aamir",
    "age": 22
}

student["city"] = "Jaipur"

print(student)
```

Output:

```
{'name': 'Aamir', 'age': 22, 'city': 'Jaipur'}
```

Dry run

Step	Code	Dictionary after execution

1	`student = {"name": "Aamir"}`	`{"name": "Aamir"}`
2	`student["age"] = 22`	`{"name": "Aamir", "age": 22}`
3	`student["city"] = "Jaipur"`	`{"name": "Aamir", "age": 22, "city": "Jaipur"}`

Every assignment using a previously absent key creates a new entry.

5. Update existing entries

Use the same syntax to update a value.

```
student = {
    "name": "Aamir",
    "marks": 70
}

student["marks"] = 95

print(student)
```

Output:

```
{'name': 'Aamir', 'marks': 95}
```

Notice that `"marks"` was not duplicated. Its old value, `70`, was replaced by `95`.

Dry run

Step	Operation	`marks` value
1	Create dictionary	70
2	`student["marks"] = 95`	95
3	`print(student["marks"])`	95

Add vs update

The same assignment syntax handles both operations.

```
student = {"name": "Aamir"}
```

```
student["age"] = 22   # Adds a new key
student["name"] = "Azad"  # Updates an existing key

print(student)
# {'name': 'Azad', 'age': 22}
```

6. Update multiple entries with `update()`

The `update()` method adds new keys and replaces existing values.

Syntax:

```
dictionary.update(other)
```

Here, `other` can be another dictionary or an iterable of key-value pairs. Keyword arguments can also be used.

```
student = {
    "name": "Aamir",
    "marks": 70
}

student.update({
    "marks": 90,
    "age": 22,
    "city": "Jaipur"
})

print(student)
```

Output:

```
{
    'name': 'Aamir',
    'marks': 90,
    'age': 22,
    'city': 'Jaipur'
}
```

Dry run

Key	Before	Update operation	After
`name`	Aamir	Not supplied	Aamir
`marks`	70	Replace with 90	90
`age`	Missing	Add 22	22
`city`	Missing	Add Jaipur	Jaipur

Another valid form:

```
student.update(age=23, city="Delhi")
```

Important: `update()` modifies the original dictionary and returns `None`.

```
result = student.update({"marks": 100})
print(result)  # None
```

Don't assign the return value of `update()` back to your dictionary.

7. Delete dictionary entries

Python provides several ways to remove entries.

A. `del` — delete a specific key

```
student = {
    "name": "Aamir",
    "age": 22,
    "city": "Jaipur"
}

del student["age"]

print(student)
# {'name': 'Aamir', 'city': 'Jaipur'}
```

If the key does not exist, `del student["missing"]` raises a `KeyError`.

B. `pop()` — remove and return a value

```
student = {
    "name": "Aamir",
    "age": 22,
    "marks": 85
}

removed_value = student.pop("age")

print(removed_value)  # 22
print(student)
# {'name': 'Aamir', 'marks': 85}
```

You can supply a default to avoid an error if the key is missing:

```
print(student.pop("city", "Not found"))
# Not found
```

C. `popitem()` — remove the last inserted entry

```
student = {"name": "Aamir", "age": 22, "marks": 85}

item = student.popitem()

print(item)     # ('marks', 85)
print(student)  # {'name': 'Aamir', 'age': 22}
```

In modern Python, `popitem()` removes the last inserted key-value pair.

D. `clear()` — remove everything

```
student = {"name": "Aamir", "age": 22}

student.clear()

print(student)  # {}
```

Compare the deletion methods

Method	Removes	Returns
`del d[key]`	Specified key	Nothing
`d.pop(key)`	Specified key	Removed value
`d.popitem()`	Last inserted pair	`(key, value)` tuple
`d.clear()`	All entries	`None`

8. Complete example: CRUD operations

CRUD stands for **Create, Read, Update, Delete**. This is a common concept in backend development and database operations.

```
# CREATE
student = {
    "name": "Aamir",
    "marks": 80
}
print("Created:", student)

# READ
print("Name:", student["name"])

# UPDATE
student["marks"] = 95
print("Updated:", student)

# ADD
student["city"] = "Jaipur"
print("Added:", student)

# DELETE
student.pop("city")
print("Deleted:", student)
```

Output:

```
Created: {'name': 'Aamir', 'marks': 80}
Name: Aamir
Updated: {'name': 'Aamir', 'marks': 95}
```

```
Added: {'name': 'Aamir', 'marks': 95, 'city': 'Jaipur'}
Deleted: {'name': 'Aamir', 'marks': 95}
```

This pattern is useful when creating APIs for student records, employee details, products, or user profiles.

---

9. Mini challenge: predict the output

Study this code carefully before revealing the answer.

```
student = {"name": "Aamir", "marks": 75}

student["age"] = 22
student["marks"] = 90
student.update({"city": "Jaipur", "age": 23})
removed = student.pop("marks")

print(student)
print(removed)
```

Show answer and dry run

10. Practice questions

Try solving these independently in Python.

Question 1 / 6 Beginner

Create a dictionary for a book with title, author and price.

Hint: Use three key-value pairs.

Reveal solution

Previous Next

11. Interview questions and answers

Q1. What is a dictionary in Python?

A dictionary is a mutable mapping that stores data as key-value pairs.

Q2. How do you add a key to a dictionary?

```
d["city"] = "Jaipur"
```

Q3. How do you update an existing key?

```
d["age"] = 23
```

If the key already exists, its value is replaced.

Q4. What is the difference between `get()` and square brackets?

`d.get("age")` returns `None` by default when the key is missing. `d["age"]` raises `KeyError`.

Q5. What is the difference between `pop()` and `del`?

`pop()` returns the removed value. `del` removes the entry without returning its value.

Q6. Does `update()` return the updated dictionary?

No. It modifies the original dictionary and returns `None`.

Q7. Can a dictionary contain duplicate keys?

No. A later assignment to an existing key overwrites its value.

Q8. What is the difference between `clear()` and `del`?

`clear()` removes all entries but keeps the dictionary object. `del d` deletes the variable binding to that dictionary; it does not necessarily destroy the object if other references exist.

12. Quick revision

- Create a dictionary with curly braces or dict().
- Read a value using d[key] or d.get(key).
- Add an entry using d[new_key] = value.
- Update an entry using d[existing_key] = new_value.

- Update several entries using d.update().
- Remove an entry using del or pop().
- Remove the last inserted pair using popitem().
- Empty a dictionary using clear().
- Explain the difference between adding and updating.

0 of 9 concepts revised.

Next lesson: Dictionary methods →

Your task before moving on: Write a small program that creates a student dictionary, adds a new subject, updates the marks, safely reads a missing key, and deletes one entry. Try doing it without copying the examples above.

You said:

Teach me Python dictionary and set topic 1: Create, access, add, update and delete dictionary entries. Explain step by step with code examples, dry runs, practice questions and interview questions.

ChatGPT said:

Python Dictionary Topic 1: Create, Access, Add, Update and Delete

Let's study this step by step, from the basics to interview-level understanding. We'll focus on **dictionary operations only** in this lesson.

1. What is a dictionary?

A Python dictionary (`dict`) stores information in **key-value pairs**.

Example: a student record.

```
student = {
    "name": "Aamir",
    "age": 22,
    "marks": 85
}

print(student)
```

Output:

```
{'name': 'Aamir', 'age': 22, 'marks': 85}
```

Think of it like a table:

Key	Value
`"name"`	`"Aamir"`
`"age"`	`22`
`"marks"`	`85`

- **Key:** The label used to identify information.
- **Value:** The information stored under that key.

Important properties:

- Keys must be unique.
- Values can be duplicated.
- Keys must be hashable.
- Dictionaries are mutable, so you can add, change, or remove entries.
- Dictionaries preserve insertion order in modern Python.

2. Create a dictionary

Method 1: Using curly braces

```
student = {
    "name": "Aamir",
    "age": 22,
    "course": "Python"
}
```

Method 2: Create an empty dictionary

```
student = {}
student["name"] = "Aamir"

print(student)
# {'name': 'Aamir'}
```

Method 3: Using `dict()`

```
student = dict(name="Aamir", age=22)

print(student)
# {'name': 'Aamir', 'age': 22}
```

Interview question: How do you create an empty dictionary?

```
d = {}
# Or
d = dict()
```

Both are correct.

3. Access dictionary values

Method A: Using square brackets `[]`

```
student = {"name": "Aamir", "age": 22, "marks": 85}

print(student["name"])
print(student["marks"])
```

Output:

```
Aamir
85
```

Dry run:

Expression	Result
`student["name"]`	`"Aamir"`
`student["age"]`	`22`
`student["marks"]`	`85`

What if the key doesn't exist?

```
print(student["city"])
```

This raises a `KeyError`.

Method B: Using `get()`

The `get()` method safely reads a value when a key might be missing.

```
student = {"name": "Aamir", "age": 22}

print(student.get("name"))              # Aamir
print(student.get("city"))              # None
print(student.get("city", "Unknown"))  # Unknown
```

Syntax:

```
dictionary.get(key, default)
```

- `key`: the key you want to access.
- `default`: optional value returned if the key is missing. By default, it is `None`.

Remember: `d[key]` raises an error for a missing key; `d.get(key)` returns `None` by default.

4. Add a new dictionary entry

Use this syntax:

```
dictionary[new_key] = value
```

Example:

```
student = {"name": "Aamir"}

student["age"] = 22
student["city"] = "Jaipur"

print(student)
```

Output:

```
{'name': 'Aamir', 'age': 22, 'city': 'Jaipur'}
```

Dry run

Step	Code	Dictionary
1	`student = {"name": "Aamir"}`	`{"name": "Aamir"}`
2	`student["age"] = 22`	`{"name": "Aamir", "age": 22}`
3	`student["city"] = "Jaipur"`	`{"name": "Aamir", "age": 22, "city": "Jaipur"}`

Each assignment to a previously absent key adds a new entry.

5. Update an existing entry

You can use the same syntax to change an existing value.

```
student = {
    "name": "Aamir",
    "marks": 70
}

student["marks"] = 95

print(student)
```

Output:

```
{'name': 'Aamir', 'marks': 95}
```

Dry run

Step	Operation	Marks
1	Create dictionary	70
2	`student["marks"] = 95`	95
3	`print(student["marks"])`	95

The key `"marks"` is not duplicated. Its value is replaced.

Add vs update

```
student = {"name": "Aamir"}

student["age"] = 22       # Add
student["name"] = "Azad"  # Update

print(student)
# {'name': 'Azad', 'age': 22}
```

Interview question: How can the same syntax add or update an entry?

Python checks whether the key already exists. If it does, the value is replaced; otherwise, a new entry is created.

6. Update multiple entries using `update()`

The `update()` method adds new entries and replaces values for existing keys.

```
student = {
    "name": "Aamir",
    "marks": 70
}

student.update({
    "marks": 90,
    "age": 22,
    "city": "Jaipur"
})

print(student)
```

Output:

```
{
    'name': 'Aamir',
    'marks': 90,
    'age': 22,
    'city': 'Jaipur'
}
```

Dry run

Key	Original value	New value
`name`	Aamir	Aamir (unchanged)
`marks`	70	90
`age`	Missing	22 (added)
`city`	Missing	Jaipur (added)

You can also write:

```
student.update(age=23, city="Delhi")
```

Common mistake: `update()` modifies the original dictionary and returns `None`.

```
result = student.update({"marks": 100})
print(result)  # None
```

7. Delete dictionary entries

Python has several deletion methods.

A. `del` — remove a specific key

```
student = {"name": "Aamir", "age": 22, "city": "Jaipur"}

del student["age"]

print(student)
# {'name': 'Aamir', 'city': 'Jaipur'}
```

If the key doesn't exist, `del` raises a `KeyError`.

B. `pop()` — remove and return the value

```
student = {"name": "Aamir", "age": 22, "marks": 85}

removed = student.pop("age")

print(removed)  # 22
print(student)
# {'name': 'Aamir', 'marks': 85}
```

To avoid an error when the key might be missing:

```
print(student.pop("city", "Not found"))
# Not found
```

C. `popitem()` — remove the last inserted pair

```
student = {"name": "Aamir", "age": 22, "marks": 85}

item = student.popitem()

print(item)     # ('marks', 85)
print(student)  # {'name': 'Aamir', 'age': 22}
```

D. `clear()` — remove all entries

```
student = {"name": "Aamir", "age": 22}

student.clear()

print(student)
# {}
```

Compare deletion methods

Method	Behavior	Return
`del d[key]`	Removes a particular key	No returned value

`d.pop(key)`	Removes a particular key	Removed value
`d.popitem()`	Removes last inserted pair	`(key, value)`
`d.clear()`	Removes all entries	`None`

8. Complete CRUD example

CRUD stands for Create, Read, Update, Delete.

```
# CREATE
student = {"name": "Aamir", "marks": 80}
print("Created:", student)

# READ
print("Name:", student["name"])

# UPDATE
student["marks"] = 95
print("Updated:", student)

# ADD
student["city"] = "Jaipur"
print("Added:", student)

# DELETE
student.pop("city")
print("Deleted:", student)
```

Output:

```
Created: {'name': 'Aamir', 'marks': 80}
Name: Aamir
Updated: {'name': 'Aamir', 'marks': 95}
Added: {'name': 'Aamir', 'marks': 95, 'city': 'Jaipur'}
Deleted: {'name': 'Aamir', 'marks': 95}
```

This pattern is useful when working with student records, employee data, user profiles, and backend APIs.

---

9. Practice questions

Try writing the code yourself before checking the solutions.

Question 1 of 6 Beginner practice

Create a dictionary for a book with title, author and price.

Hint: Use three key-value pairs.

Show solution

Previous Next

10. Interview questions and answers

Q1. What is a dictionary in Python?

A mutable mapping that stores key-value pairs.

Q2. Can a dictionary have duplicate keys?

No. Assigning the same key again replaces its value.

Q3. What is the difference between `get()` and `[]`?

`get()` returns a default value when a key is missing. Square brackets raise `KeyError`.

Q4. What is the difference between `pop()` and `del`?

`pop()` returns the removed value, whereas `del` does not return the removed entry.

Q5. Does `update()` return the updated dictionary?

No. It modifies the dictionary in place and returns `None`.

Q6. What does `clear()` do?

It removes all entries but leaves the dictionary object available.

Q7. How do you add multiple entries at once?

Use `update()` with a dictionary or iterable of key-value pairs.

Q8. Can dictionary values be duplicated?

Yes. Different keys can have the same value.

11. Mini test: predict the output

```
student = {"name": "Aamir", "marks": 75}

student["age"] = 22
student["marks"] = 90
student.update({"city": "Jaipur", "age": 23})
removed = student.pop("marks")

print(student)
print(removed)
```

Reveal answer and dry run

12. Your revision checklist

- [ ] Create a dictionary using {} and dict().
- [ ] Access values with [] and get().
- [ ] Add new entries using assignment.
- [ ] Update existing entries using assignment.
- [ ] Update multiple entries with update().
- [ ] Delete entries using del, pop(), popitem() and clear().
- [ ] Explain CRUD and predict dictionary outputs.

0/7 completed

Continue to Topic 2: Dictionary methods →

Before moving on: Write one program that creates a student dictionary, adds a subject, updates the marks, safely reads a missing key, and deletes one entry. Try to do it without copying the examples.

You said:

complete set

ChatGPT said:

Python Set — Complete Guide (Basics to Advanced)

A **set in Python** is a collection used to store unique elements. It is especially useful when you need to remove duplicates, check membership quickly, or perform mathematical operations such as union, intersection, and difference.

This guide covers Python sets step by step, with beginner-friendly definitions, code examples, dry runs, common mistakes, coding interview questions, and practice exercises.

1. What is a set in Python?

A set is an **unordered collection of unique elements**.

```
numbers = {10, 20, 30, 20, 10}

print(numbers)
```

Output — the order may vary:

```
{10, 20, 30}
```

The duplicate values `20` and `10` are automatically removed.

Remember these 4 properties

Unique elements

Duplicates are not stored.

Unordered

Do not rely on a particular display order or use positional indexing.

Mutable

You can add or remove elements after creating a set.

Elements must be hashable

Integers, strings and tuples of hashable elements can be set elements. Lists and dictionaries cannot.

2. Creating a set

Method 1: Using curly braces `{}`

```
fruits = {"apple", "banana", "mango"}
print(fruits)
```

Output: a set containing the three fruits, possibly in a different order.

Method 2: Using `set()`

```
numbers = set([1, 2, 2, 3, 3, 4])
print(numbers)
```

Output:

```
{1, 2, 3, 4}
```

You can create a set from a list, tuple, string, or another iterable.

Method 3: Creating an empty set

```
a = set()
print(a)
print(type(a))
```

Output:

```
set()
<class 'set'>
```

Important interview question: Why is `{}` not an empty set?

```
a = {}
print(type(a))
```

Output:

```
<class 'dict'>
```

`{}` creates an empty dictionary. Use `set()` to create an empty set.

Dry run: Removing duplicates

```
values = [5, 5, 10, 10, 15]
unique_values = set(values)
print(unique_values)
```

Step	Operation	Unique values
1	Read `5`	`{5}`
2	Read another `5`	`{5}`
3	Read `10`	`{5, 10}`
4	Read another `10`	`{5, 10}`
5	Read `15`	`{5, 10, 15}`

The order in the output is not guaranteed.

3. Accessing set elements

Unlike lists and tuples, sets do **not support indexing or slicing**.

```
numbers = {10, 20, 30}

print(numbers[0])  # TypeError
```

Why? A set has no positional indexing such as index `0` or index `1`.

How do you access elements?

Use a `for` loop:

```
numbers = {10, 20, 30}
```

```
for number in numbers:
    print(number)
```

Output contains `10`, `20`, and `30`, but the order is not guaranteed.

Check whether an element exists

Use the `in` operator.

```
fruits = {"apple", "banana", "mango"}

print("apple" in fruits)
print("grapes" in fruits)
```

Output:

```
True
False
```

Use `not in` to check that an element is absent:

```
print("grapes" not in fruits)  # True
```

Interview tip: Set membership testing is typically $O(1)$ on average, making sets useful for checking whether an item exists in a large collection.

---

4. Adding elements to a set

`add()` — add one element

Syntax:

```
set_name.add(element)
```

Example:

```
colors = {"red", "blue"}
```

```
colors.add("green")
print(colors)
```

Result: `red`, `blue`, and `green` are present.

Adding an element that already exists does not create a duplicate:

```
colors.add("red")
print(colors)
```

The set remains unchanged.

Dry run of `add()`

```
numbers = {1, 2}
numbers.add(3)
numbers.add(2)
```

Step	Code	Set contents
1	Initial set	`{1, 2}`
2	`add(3)`	`{1, 2, 3}`
3	`add(2)`	`{1, 2, 3}`

`update()` — add multiple elements

Syntax:

```
set_name.update(iterable)
```

Example:

```
numbers = {1, 2}

numbers.update([3, 4, 5])
print(numbers)
```

Result: `{1, 2, 3, 4, 5}`

You can pass multiple iterables:

```
numbers = {1, 2}

numbers.update([3, 4], (5, 6), {7, 8})
print(numbers)
```

All elements from the supplied iterables are added. Duplicates are ignored.

Be careful: if you pass a string, Python adds its characters individually.

```
letters = {"a"}
letters.update("cat")

print(letters)
```

The set contains the unique characters `a`, `c`, and `t`.

Difference between `add()` and `update()`:

`add()`	`update()`
Adds one element	Adds elements from one or more iterables
`s.add((1, 2))` adds one tuple element	`s.update((1, 2))` adds `1` and `2` separately

---

5. Removing elements from a set

Python provides several removal methods. The important difference is what happens when the requested element does not exist.

`remove()` — raises an error if absent

```
numbers = {10, 20, 30}

numbers.remove(20)
print(numbers)
```

Result: `{10, 30}`

But this raises `KeyError`:

```
numbers.remove(100)
```

Use `remove()` when you expect the element to exist and want a missing element to be treated as an error.

`discard()` **— does not raise an error if absent**

```
numbers = {10, 20, 30}

numbers.discard(20)
numbers.discard(100)

print(numbers)
```

Result: `{10, 30}`

`discard(100)` does nothing because `100` is absent.

`pop()` **— removes and returns an arbitrary element**

```
numbers = {10, 20, 30}

removed = numbers.pop()

print("Removed:", removed)
print("Remaining:", numbers)
```

`pop()` removes and returns one element, but you should **not assume which element it will return**.

If the set is empty, `pop()` raises `KeyError`.

`clear()` **— removes every element**

```
numbers = {1, 2, 3}
numbers.clear()

print(numbers)
```

Output:

```
set()
```

Dry run: Removal methods

```
s = {1, 2, 3, 4}

s.remove(2)
s.discard(10)
print(s)

s.clear()
print(s)
```

Step	Operation	Result
1	Initial set	`{1, 2, 3, 4}`
2	`remove(2)`	`{1, 3, 4}`
3	`discard(10)`	Unchanged
4	`clear()`	`set()`

Quick comparison

Method	Missing element	Return value
`remove(x)`	Raises `KeyError`	`None` on success
`discard(x)`	No error	`None`
`pop()`	Empty set raises `KeyError`	Removed element
`clear()`	Not applicable	`None`

---

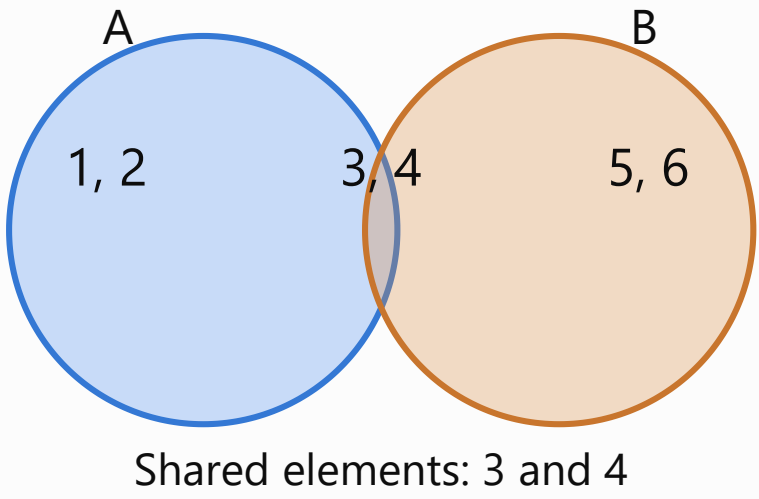
6. Mathematical operations on sets

This is one of the most important topics for interviews and coding problems.

Imagine two sets:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

Visualize the two sets



The overlapping region represents the intersection. The full area of both circles represents the union.

6.1 Union — combine all unique elements

Union returns every element found in either set.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B)
print(A.union(B))
```

Result:

```
{1, 2, 3, 4, 5, 6}
```

Symbol: `|`

Method: `union()`

6.2 Intersection — common elements

Intersection returns elements that occur in both sets.

```
print(A & B)
print(A.intersection(B))
```

Result:

```
{3, 4}
```

Symbol: `&`

Method: `intersection()`

6.3 Difference — elements in the first set but not the second

```
print(A - B)
print(B - A)
```

Results:

```
{1, 2}
{5, 6}
```

Symbol: `-`

Method: `difference()`

Important: Difference is directional. `A - B` and `B - A` usually differ.

6.4 Symmetric difference — elements in either set, but not both

```
print(A ^ B)
print(A.symmetric_difference(B))
```

Result:

```
{1, 2, 5, 6}
```

Symbol: `^`

Method: `symmetric_difference()`

Operation summary

Operation	Symbol	Result for A and B
Union	`A | B`	`{1, 2, 3, 4, 5, 6}`
Intersection	`A & B`	`{3, 4}`
Difference	`A - B`	`{1, 2}`
Symmetric difference	`A ^ B`	`{1, 2, 5, 6}`

Dry run: Find common students

```
python_students = {"Aman", "Riya", "Kabir"}
java_students = {"Riya", "Kabir", "Sara"}

common = python_students & java_students
print(common)
```

Student	In Python set?	In Java set?	In intersection?
Aman	Yes	No	No
Riya	Yes	Yes	Yes
Kabir	Yes	Yes	Yes
Sara	No	Yes	No

Result: `{"Riya", "Kabir"}`

7. Updating a set using mathematical operations

The following methods change the original set instead of returning a separate result.

```
A = {1, 2, 3}
B = {3, 4, 5}
```

```
A.update(B)
print(A)
```

Result: `{1, 2, 3, 4, 5}`

Here are the main in-place methods:

Method	What it does
`update(other)`	Adds all elements from other iterables
`intersection_update(other)`	Keeps only common elements
`difference_update(other)`	Removes elements found in the other set
`symmetric_difference_update(other)`	Keeps elements found in only one set

Examples:

```
# Keep only common elements
A = {1, 2, 3}
A.intersection_update({2, 3, 4})
print(A)  # {2, 3}

# Remove elements that occur in another set
A = {1, 2, 3}
A.difference_update({2, 3})
print(A)  # {1}

# Keep elements unique to either set
A = {1, 2, 3}
A.symmetric_difference_update({3, 4})
print(A)  # {1, 2, 4}
```

Interview point: In-place update methods modify the set and return `None`. Do not assign their return value back to the set.

Incorrect:

```
A = {1, 2}
A = A.update({3})  # A becomes None
```

Correct:

```
A = {1, 2}
A.update({3})
print(A)  # {1, 2, 3}
```

8. Set comparison: subset, superset and disjoint

These methods are frequently asked in interviews.

`issubset()` — is every element present in another set?

```
A = {1, 2}
B = {1, 2, 3, 4}

print(A.issubset(B))  # True
print(A <= B)         # True
```

`A` is a subset of `B` because every element of `A` exists in `B`.

`issuperset()` — does this set contain every element of another set?

```
print(B.issuperset(A))  # True
print(B >= A)          # True
```

`isdisjoint()` — do the sets have no common elements?

```
A = {1, 2}
B = {3, 4}

print(A.isdisjoint(B))  # True
```

If even one element is shared, `isdisjoint()` returns `False`.

```
print({1, 2}.isdisjoint({2, 3}))  # False
```

Quick memory trick

- **Subset:** A is inside B.
- **Superset:** A contains B.
- **Disjoint:** No overlap.

9. Looping through a set

Basic `for` loop

```
numbers = {10, 20, 30}

for number in numbers:
    print(number)
```

All three values are printed, but their order is not guaranteed.

Loop with a condition

```
numbers = {1, 2, 3, 4, 5, 6}

for number in numbers:
    if number % 2 == 0:
        print(number)
```

This prints the even numbers: `2`, `4`, and `6`, in an unspecified order.

Create a list from a set

```
numbers = {3, 1, 2}
result = sorted(numbers)

print(result)
```

Output:

```
[1, 2, 3]
```

Remember the difference:

- `sorted(set_name)` returns a **list** in sorted order.
- `set_name` itself remains a set and has no guaranteed positional order.

10. Set comprehension

Set comprehension creates a set using a compact expression.

Syntax:

```
{expression for item in iterable if condition}
```

The `if condition` part is optional.

Example 1: Squares of numbers

```
squares = {x ** 2 for x in range(1, 6)}
print(squares)
```

Result: a set containing `1, 4, 9, 16, 25`.

Dry run:

`x`	`x ** 2`	Added value
1	1	1
2	4	4
3	9	9
4	16	16
5	25	25

Example 2: Even numbers

```
even_numbers = {x for x in range(1, 11) if x % 2 == 0}
print(even_numbers)
```

Result: `{2, 4, 6, 8, 10}`

Example 3: Unique characters in a string

```
word = "banana"
unique_letters = {ch for ch in word}

print(unique_letters)
```

Result: the unique characters `b`, `a`, and `n`.

Set comprehension vs list comprehension

```
list_result = [x % 3 for x in range(6)]
set_result = {x % 3 for x in range(6)}

print(list_result)
print(set_result)
```

The list preserves duplicates and order; the set removes duplicates and does not guarantee order.

---

11. Important set rules: mutable vs hashable

A set is mutable, but its elements must be **hashable**. Hashability means an object has a stable hash value and can be used as a set element or dictionary key.

Allowed elements

```
s = {10, 3.5, "hello", (1, 2)}
print(s)
```

Integers, floats, strings, and tuples made up of hashable values can be elements.

Not allowed: list or dictionary

```
s = {[1, 2, 3]}  # TypeError


s = {{'a': 1}}   # TypeError
```

Lists and dictionaries are mutable and unhashable, so they cannot be set elements.

What about tuples?

A tuple is acceptable only if all its elements are hashable.

```
valid = {(1, 2), (3, 4)}
print(valid)
```

But this fails:

```
invalid = {(1, [2, 3])}  # TypeError
```

The tuple contains a list, which is unhashable.

Common interview trap: An immutable container is not automatically hashable. Its contents matter too.

12. `frozenset` — an immutable set

A `frozenset` behaves like a set, but it cannot be modified after creation.

```
numbers = frozenset([1, 2, 2, 3])

print(numbers)
```

It contains the unique values `1`, `2`, and `3`.

You can perform operations such as union and intersection:

```
A = frozenset([1, 2, 3])
B = frozenset([3, 4, 5])

print(A | B)
print(A & B)
```

But you cannot add or remove elements:

```
A.add(4)  # AttributeError
```

A `frozenset` is hashable when its elements are hashable, so it can be used as a dictionary key or as an element inside another set.

```
data = {
    frozenset({1, 2}): "pair"
}

print(data[frozenset({1, 2})])  # pair
```

Set vs frozenset

Feature	`set`	`frozenset`
Mutable	Yes	No
Add/remove elements	Yes	No
Mathematical operations	Yes	Yes
Can be a dictionary key	No	Yes, if hashable
Can contain duplicates	No	No

13. Set vs list vs tuple vs dictionary

Feature	Set	List	Tuple	Dictionary
Syntax	`{1, 2}`	`[1, 2]`	`(1, 2)`	`{"a": 1}`
Stores	Unique values	Values	Values	Key-value pairs
Duplicates	No	Yes	Yes	Keys must be unique
Ordered	No positional order guaranteed	Yes	Yes	Insertion-ordered
Indexing	No	Yes	Yes	By key
Mutable	Yes	Yes	No	Yes
Common use	Uniqueness, membership, set operations	General sequence	Fixed sequence	Lookup by key

Remember: a dictionary also uses curly braces, but it stores key-value pairs.

```
a = {1, 2, 3}           # Set
b = {"x": 10, "y": 20}  # Dictionary
c = {}                   # Empty dictionary
d = set()                # Empty set
```

14. Set coding interview questions with answers and dry runs

Question 1: Remove duplicates from a list

```
numbers = [1, 2, 2, 3, 4, 4, 5]

unique = set(numbers)

print(unique)
```

Output: `{1, 2, 3, 4, 5}`

Dry run: Each element is inserted into the set only if it is not already present. The repeated `2` and `4` do not create new elements.

If you need to preserve the original order of first occurrences, use a different approach:

```
numbers = [1, 2, 2, 3, 4, 4, 5]

unique = list(dict.fromkeys(numbers))
print(unique)
```

Output:

```
[1, 2, 3, 4, 5]
```

Question 2: Find common elements in two lists

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]
```

```
common = set(a) & set(b)
print(common)
```

Output: `{3, 4}`

Dry run:

1. Convert `a` to `{1, 2, 3, 4}`.
2. Convert `b` to `{3, 4, 5, 6}`.
3. Find the intersection.
4. Keep `3` and `4`, because both sets contain them.

Question 3: Find elements that occur in only one list

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]

result = set(a) ^ set(b)
print(result)
```

Output: `{1, 2, 5, 6}`

The symmetric difference removes elements shared by both sets.

Question 4: Check whether a list contains duplicates

```
numbers = [1, 2, 3, 2, 4]

if len(numbers) != len(set(numbers)):
    print("Duplicates found")
else:
    print("No duplicates")
```

Output:

```
Duplicates found
```

Why does this work?

- Original list length: `5`
- Unique set length: `4`

- Because the lengths differ, the list contains duplicates.

Question 5: Find all duplicate values

```
numbers = [1, 2, 3, 2, 4, 1, 5, 3]

seen = set()
duplicates = set()

for number in numbers:
    if number in seen:
        duplicates.add(number)
    else:
        seen.add(number)

print(duplicates)
```

Output:

```
{1, 2, 3}
```

Dry run:

Current number	`seen` before checking	Action	`duplicates`
1	`{}`	Add to `seen`	`{}`
2	`{1}`	Add to `seen`	`{}`
3	`{1, 2}`	Add to `seen`	`{}`
2	`{1, 2, 3}`	Already seen	`{2}`
4	`{1, 2, 3}`	Add to `seen`	`{2}`
1	`{1, 2, 3, 4}`	Already seen	`{1, 2}`
5	`{1, 2, 3, 4}`	Add to `seen`	`{1, 2}`
3	`{1, 2, 3, 4, 5}`	Already seen	`{1, 2, 3}`

Question 6: Find unique characters in a string

```
text = "programming"

unique_chars = set(text)

print(unique_chars)
print(len(unique_chars))
```

The set contains each distinct character once. The number of unique characters is `8`.

Question 7: Check whether two strings are anagrams

Anagrams contain the same letters with the same frequencies. A set alone is insufficient because it ignores frequencies.

For example, `"aab"` and `"abb"` have the same set of letters but are not anagrams.

Correct solution:

```
def are_anagrams(a, b):
    return sorted(a) == sorted(b)

print(are_anagrams("listen", "silent"))  # True
print(are_anagrams("hello", "world"))    # False
```

Question 8: Find missing numbers between two sets

```
expected = {1, 2, 3, 4, 5, 6}
actual = {1, 2, 4, 6}

missing = expected - actual
print(missing)
```

Output: `{3, 5}`

Question 9: Find elements present in every set

```
a = {1, 2, 3, 4}
b = {2, 3, 4, 5}
c = {0, 2, 3, 8}
```

```
common = a & b & c
print(common)
```

Output:

```
{2, 3}
```

Question 10: Check if one set is a subset of another

```
required = {"Python", "SQL"}
skills = {"Python", "SQL", "Excel", "Pandas"}

if required.issubset(skills):
    print("All required skills are present")
else:
    print("Some skills are missing")
```

Output:

```
All required skills are present
```

Question 11: Count unique words in a sentence

```
sentence = "python is easy and python is useful"

words = sentence.lower().split()
unique_words = set(words)

print(unique_words)
print("Unique count:", len(unique_words))
```

Output contains six unique words: `python`, `is`, `easy`, `and`, `useful` — actually **five** unique words.

The count is `5`.

Question 12: Find the union of multiple sets

```
a = {1, 2}
b = {2, 3}
c = {3, 4}

result = a.union(b, c)
print(result)
```

Output:

```
{1, 2, 3, 4}
```

Question 13: Find the largest number in a set

```
numbers = {12, 5, 90, 33, 18}

print(max(numbers))
```

Output: `90`

Similarly, use `min(numbers)` for the smallest value and `sum(numbers)` for the total.

Question 14: Create a set of squares of even numbers

```
squares = {x ** 2 for x in range(1, 11) if x % 2 == 0}

print(squares)
```

Output: `{4, 16, 36, 64, 100}`

Question 15: Find the second-largest distinct number

```
numbers = [10, 20, 20, 30, 40, 40]

unique = set(numbers)
unique.remove(max(unique))

print(max(unique))
```

Output: `30`

This solution assumes there are at least two distinct values. If not, handle that case before calling `max()` again.

---

15. Set interview questions and answers

Beginner questions

1. What is a set in Python?

A set is an unordered collection of unique, hashable elements. It is mutable.

2. Why do we use sets?

To remove duplicates, test membership, find common values, compare collections, and perform union or difference operations.

3. How do you create an empty set?

```
s = set()
```

`{}` creates an empty dictionary instead.

4. Can a set contain duplicates?

No. Duplicate values are automatically removed.

5. Can we access a set using an index?

No. Sets do not support indexing or slicing. Use iteration or membership testing.

6. Can a set contain a list?

No. Lists are unhashable. A tuple containing only hashable elements can be a set element.

7. Is a set mutable?

Yes. You can add and remove elements.

8. What is the difference between `add()` and `update()`?

`add()` inserts one element; `update()` inserts elements from one or more iterables.

9. What is the difference between `remove()` and `discard()`?

`remove()` raises `KeyError` if the element is missing; `discard()` does not.

10. What is a `frozenset`?

An immutable set whose elements cannot be added or removed after creation.

Intermediate questions

11. What is the difference between union and intersection?

Union combines unique elements from both sets. Intersection returns only shared elements.

12. What is the difference between difference and symmetric difference?

- Difference: elements in the first set but not the second.
- Symmetric difference: elements in either set but not both.

13. What is the difference between `pop()` and `remove()`?

`pop()` removes and returns an arbitrary element. `remove(x)` removes a specified element and returns `None` on success.

14. What is set comprehension?

A compact way to create a set from an iterable, optionally applying a condition.

```
evens = {x for x in range(10) if x % 2 == 0}
```

15. Can a set contain a tuple?

Yes, if all elements inside the tuple are hashable.

16. Can a set be a dictionary key?

No, because a normal set is mutable and unhashable. A suitable `frozenset` can be used as a dictionary key.

17. What is the average time complexity of membership testing?

Typically $O(1)$ on average, although worst-case behavior can be $O(n)$.

18. Does converting a list to a set preserve order?

No. Do not rely on set order. Use `sorted()` if sorted output is required or an order-preserving approach if original order matters.

19. Can a set contain `True` and `1` separately?

No. In Python, `True == 1` and they have compatible hash values, so they represent the same set element.

```
print({True, 1, False, 0})
```

The set contains two distinct values: one equivalent to `True` / `1` and one equivalent to `False` / `0`.

20. Does `update()` return the updated set?

No. It modifies the set in place and returns `None`.

16. Time complexity cheat sheet

For a set with n elements:

Operation	Typical average complexity
Add an element	$O(1)$
Remove/discard an element	$O(1)$
Membership: `x in s`	$O(1)$
Iterate through the set	$O(n)$
Copy the set	$O(n)$
Union of two sets	$O(n+m)$ typically
Intersection	Depends on implementation and relative sizes; typically efficient

Here, n and m are the sizes of the two sets. These are typical complexities, not guarantees for every possible input.

17. Practice questions — test yourself

Try these without looking at the answers first.

Set knowledge quiz

0/7 answered

1. Create a set containing 10, 20, 20, 30, 30, 40. How many unique elements are there?

- 4
- 6
- 5

---

2. Which statement creates an empty set?

- s = {}
- s = set()
- s = []

---

3. What does {1, 2, 3} & {2, 3, 4} return?

- {1, 2, 3, 4}
- {2, 3}
- {1, 4}

---

4. Which method safely ignores a missing element?

- remove()
- pop()
- discard()

5. What is the type of the result of sorted({3, 1, 2})?

- set
- tuple
- list

---

6. Which can be a set element?

- [1, 2]
- {'a': 1}
- (1, 2)

7. What does {1, 2, 3} ^ {3, 4} return?

- {3}
- {1, 2, 4}
- {1, 2, 3, 4}

Reset quiz

18. Final revision checklist

Your revision progress 0%

- Define a set and explain its four main properties.
- Create an empty set and a set from a list.
- Explain why sets do not support indexing.
- Use add() and update().
- Explain remove(), discard(), pop(), and clear().
- Perform union, intersection, difference, and symmetric difference.
- Use subset, superset, and disjoint checks.
- Write a set comprehension.
- Explain hashable elements and frozenset.
- Remove duplicates and find common elements in lists.
- Detect duplicate values and find the second-largest distinct value.
- Explain average membership-test complexity.

Recommended next step: Practice set coding problems until you can solve them without notes. Then study dictionary and set combinations, such as finding word frequencies, removing duplicates while preserving order, and comparing two datasets.

You said:

complete dictionary

ChatGPT said:

Python Dictionary — Complete Guide (Basics to Advanced)

A **dictionary in Python** stores data as **key-value pairs**. It is one of the most important Python concepts for coding interviews, backend development, and data science.

For example, think of a student record:

Real-world example: Student record

Key	Value
`"name"`	`"Aman"`
`"age"`	`21`
`"marks"`	`92`

The key identifies the information; the value stores that information.

1. What is a dictionary?

A dictionary is a mutable, insertion-ordered collection that maps unique keys to values.

```
student = {
    "name": "Aman",
    "age": 21,
    "marks": 92
}

print(student)
```

Output:

```
{'name': 'Aman', 'age': 21, 'marks': 92}
```

Important properties

- **Key-value pairs:** Every entry connects a key to a value.
- **Unique keys:** Duplicate keys do not create separate entries; the last assigned value wins.
- **Insertion-ordered:** Python preserves the order in which keys are first inserted.
- **Mutable:** You can add, update, and delete entries.
- **Keys must be hashable:** Strings, integers, and suitable tuples can be keys; lists and dictionaries cannot.
- **Values can be any type:** Numbers, strings, lists, other dictionaries, and even sets can be values.

Example:

```
student = {
    "name": "Aman",
    "skills": ["Python", "SQL"],
    "marks": {"math": 90, "science": 85}
}
```

Here, a list and another dictionary are stored as values.

2. Creating a dictionary

Method 1: Using curly braces `{}`

```
person = {
    "name": "Riya",
    "age": 22,
    "city": "Delhi"
}
```

Method 2: Using `dict()`

```
person = dict(name="Riya", age=22, city="Delhi")
print(person)
```

Output:

```
{'name': 'Riya', 'age': 22, 'city': 'Delhi'}
```

This form requires valid Python identifier names for the keyword arguments.

Method 3: Creating an empty dictionary

```
data = {}
# Or:
data = dict()

print(type(data))
```

Output:

```
<class 'dict'>
```

Method 4: Creating from pairs

```
pairs = [("name", "Riya"), ("age", 22)]
person = dict(pairs)

print(person)
```

Output:

```
{'name': 'Riya', 'age': 22}
```

Method 5: Using `fromkeys()`

Creates keys with the same initial value.

```
keys = ["name", "age", "city"]
person = dict.fromkeys(keys, None)

print(person)
```

Output:

```
{'name': None, 'age': None, 'city': None}
```

Important: Avoid using a shared mutable value with `fromkeys()` when you expect independent lists or dictionaries.

```
data = dict.fromkeys(["a", "b"], [])
data["a"].append(10)

print(data)  # Both values refer to the same list
```

Both entries will show `[10]`. Create independent lists using a comprehension instead:

```
data = {key: [] for key in ["a", "b"]}
```

3. Accessing dictionary values

Method 1: Square brackets `[]`

```
student = {"name": "Aman", "age": 21}

print(student["name"])
print(student["age"])
```

Output:

```
Aman
21
```

If the key does not exist, square brackets raise a `KeyError`.

```
print(student["marks"])  # KeyError
```

Method 2: `get()`

`get()` retrieves a value without raising a `KeyError` when the key is missing.

```
student = {"name": "Aman", "age": 21}

print(student.get("name"))
print(student.get("marks"))
print(student.get("marks", 0))
```

Output:

```
Aman
None
0
```

`[]` vs `get()`

Feature	`d[key]`	`d.get(key, default)`
Existing key	Returns value	Returns value
Missing key	Raises `KeyError`	Returns `None` or default
Adds a key?	No	No
Best for	Required keys	Optional keys

Dry run

```
student = {"name": "Aman", "marks": 90}

result = student.get("age", 18)
print(result)
```

Step	Action	Result
1	Search for `"age"`	Key is missing
2	Use default `18`	`result = 18`
3	Print result	`18`

Note: `get()` does not insert `"age": 18` into the dictionary.

4. Adding and updating dictionary entries

Python uses the same assignment syntax for adding a new key and updating an existing key.

Add a new entry

```
student = {"name": "Aman", "age": 21}

student["city"] = "Delhi"

print(student)
```

Output:

```
{'name': 'Aman', 'age': 21, 'city': 'Delhi'}
```

Update an existing entry

```
student = {"name": "Aman", "age": 21}

student["age"] = 22

print(student)
```

Output:

```
{'name': 'Aman', 'age': 22}
```

The existing `"age"` value is replaced; a second `"age"` entry is not created.

Dry run

```
marks = {"Python": 80, "SQL": 75}

marks["Python"] = 95
marks["Java"] = 88
```

Step	Operation	Dictionary

1	Initial	`{"Python": 80, "SQL": 75}`
2	Update Python	`{"Python": 95, "SQL": 75}`
3	Add Java	`{"Python": 95, "SQL": 75, "Java": 88}`

Updating multiple entries with `update()`

```
student = {"name": "Aman", "age": 21}

student.update({"age": 22, "city": "Delhi"})

print(student)
```

Output:

```
{'name': 'Aman', 'age': 22, 'city': 'Delhi'}
```

`update()` can accept another dictionary, key-value pairs, or keyword arguments.

```
student.update([("marks", 90), ("grade", "A")])
student.update(city="Jaipur")

print(student)
```

Remember: `update()` modifies the original dictionary and returns `None`.

Incorrect:

```
student = student.update({"age": 23})  # student becomes None
```

Correct:

```
student.update({"age": 23})
```

---

5. Important dictionary methods

These are among the most frequently asked methods in Python interviews.

5.1 `get()` — safely retrieve a value

```
user = {"name": "Riya", "age": 22}

print(user.get("name"))          # Riya
print(user.get("email"))         # None
print(user.get("email", "N/A"))  # N/A
```

It does not change the dictionary.

5.2 `setdefault()` — get a value or insert a default

```
student = {"name": "Aman"}

result = student.setdefault("age", 21)

print(result)
print(student)
```

Output:

```
21
{'name': 'Aman', 'age': 21}
```

If the key already exists, `setdefault()` returns its current value without replacing it.

```
student = {"age": 20}

result = student.setdefault("age", 25)

print(result)   # 20
print(student)  # {'age': 20}
```

Difference between `get()` and `setdefault()`:

```
d = {"a": 10}

d.get("b", 0)
print(d)  # {'a': 10}
```

```
d.setdefault("c", 0)
print(d)  # {'a': 10, 'c': 0}
```

- `get()` reads a value.
- `setdefault()` reads a value and inserts the default if the key is absent.

5.3 `update()` — add or overwrite multiple entries

```
d = {"a": 1, "b": 2}

d.update({"b": 20, "c": 3})

print(d)
```

Output:

```
{'a': 1, 'b': 20, 'c': 3}
```

5.4 `pop()` — remove a specified key and return its value

```
student = {"name": "Aman", "age": 21, "marks": 90}

removed = student.pop("age")

print(removed)
print(student)
```

Output:

```
21
{'name': 'Aman', 'marks': 90}
```

If the key is missing, `pop()` raises `KeyError` unless you provide a default.

```
print(student.pop("city", "Not found"))
```

Output:

```
Not found
```

5.5 `popitem()` — remove and return the last inserted pair

```
student = {"name": "Aman", "age": 21, "marks": 90}

result = student.popitem()

print(result)
print(student)
```

Output:

```
('marks', 90)
{'name': 'Aman', 'age': 21}
```

In modern Python, `popitem()` removes the last inserted entry. It raises `KeyError` if the dictionary is empty.

5.6 `keys()` — view all keys

```
student = {"name": "Aman", "age": 21}

print(student.keys())
```

Result: a dictionary-keys view containing `"name"` and `"age"`.

5.7 `values()` — view all values

```
print(student.values())
```

Result: a dictionary-values view containing `"Aman"` and `21`.

5.8 `items()` — view key-value pairs

```
print(student.items())
```

Result: a dictionary-items view containing `("name", "Aman")` and `("age", 21)`.

These are dynamic views: they reflect changes to the dictionary.

5.9 `clear()` — remove all entries

```
d = {"a": 1, "b": 2}
d.clear()

print(d)
```

Output:

```
{}
```

5.10 `copy()` — create a shallow copy

```
original = {"name": "Aman", "age": 21}
new_dict = original.copy()

new_dict["age"] = 25

print(original)  # age is 21
print(new_dict)  # age is 25
```

This creates a new outer dictionary, but nested mutable objects are still shared. Deep copying is covered later.

Quick revision table

Method	Purpose
`get()`	Read a value safely
`setdefault()`	Read a value or insert a default
`update()`	Add or update multiple entries
`pop()`	Remove a specified key and return its value
`popitem()`	Remove and return the last inserted pair

`keys()`	View keys
`values()`	View values
`items()`	View key-value pairs
`clear()`	Remove all entries
`copy()`	Make a shallow copy
`fromkeys()`	Create a dictionary from keys and a common value

---

6. Deleting dictionary entries

There are three common ways to remove entries.

Using `del`

```
student = {"name": "Aman", "age": 21, "marks": 90}

del student["age"]

print(student)
```

Output:

```
{'name': 'Aman', 'marks': 90}
```

If the key is absent, `del` raises `KeyError`.

Using `pop()`

```
student = {"name": "Aman", "age": 21}

age = student.pop("age")
print(age)
print(student)
```

Use `pop()` when you also need the removed value.

Using `clear()`

```
student.clear()
print(student)
```

Output: `{}`

Interview comparison:

- `del d[key]` : deletes the key, returns no value.
- `d.pop(key)` : deletes the key and returns its value.
- `d.clear()` : deletes every entry.

---

7. Looping through a dictionary

Method 1: Loop through keys

```
student = {"name": "Aman", "age": 21, "marks": 90}

for key in student:
    print(key)
```

Output:

```
name
age
marks
```

By default, looping through a dictionary gives you its keys.

Method 2: Loop through values

```
for value in student.values():
    print(value)
```

Output:

```
Aman
21
90
```

Method 3: Loop through keys and values with `items()`

```
for key, value in student.items():
    print(key, ":", value)
```

Output:

```
name : Aman
age : 21
marks : 90
```

Dry run

```
prices = {"apple": 50, "banana": 20, "mango": 80}

for fruit, price in prices.items():
    if price > 30:
        print(fruit, price)
```

Iteration	Fruit	Price	Condition	Printed?
1	apple	50	`50 > 30`	Yes
2	banana	20	`20 > 30`	No
3	mango	80	`80 > 30`	Yes

Output:

```
apple 50
mango 80
```

---

8. Checking whether a key exists

Use the `in` and `not in` operators.

```
student = {"name": "Aman", "age": 21}

print("name" in student)      # True
print("marks" in student)     # False
print("marks" not in student) # True
```

Important: Membership checks keys, not values.

```
student = {"name": "Aman", "age": 21}

print("name" in student.values())  # False
print(21 in student.values())      # True
```

For keys, `key in student` is generally the clearest approach.

9. Dictionary comprehension

Dictionary comprehension creates a dictionary using a compact expression.

Syntax:

```
{key_expression: value_expression for item in iterable}
```

You can add an optional condition:

```
{key_expression: value_expression for item in iterable if condition}
```

Example 1: Create squares

```
squares = {x: x ** 2 for x in range(1, 6)}

print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Dry run:

`x`	`x ** 2`	Key-value pair
1	1	`1: 1`
2	4	`2: 4`
3	9	`3: 9`
4	16	`4: 16`
5	25	`5: 25`

Example 2: Filter even numbers

```
even_squares = {
    x: x ** 2
    for x in range(1, 11)
    if x % 2 == 0
}

print(even_squares)
```

Output:

```
{2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

Example 3: Convert two lists into a dictionary

```
names = ["Aman", "Riya", "Kabir"]
marks = [85, 92, 78]

result = {names[i]: marks[i] for i in range(len(names))}

print(result)
```

Output:

```
{'Aman': 85, 'Riya': 92, 'Kabir': 78}
```

A more concise alternative is `zip()`:

```
result = dict(zip(names, marks))
print(result)
```

Example 4: Reverse keys and values

```
original = {"a": 1, "b": 2, "c": 3}

reversed_dict = {value: key for key, value in original.items()}

print(reversed_dict)
```

Output:

```
{1: 'a', 2: 'b', 3: 'c'}
```

This works when the original values are hashable and unique. If values repeat, some keys will be overwritten.

---

10. Nested dictionaries

A nested dictionary is a dictionary containing another dictionary as a value.

Example

```
students = {
    "student1": {
        "name": "Aman",
        "age": 21,
        "marks": 90
    },
    "student2": {
        "name": "Riya",
        "age": 22,
        "marks": 95
```

```
    }
}
```

Access nested values

```
print(students["student1"]["name"])
print(students["student2"]["marks"])
```

Output:

```
Aman
95
```

Read it from the outside inward:

`students` → `"student1"` → `"name"` → `"Aman"`

Update a nested value

```
students["student1"]["marks"] = 96
```

Add a nested entry

```
students["student1"]["city"] = "Delhi"
```

Loop through nested dictionaries

```
for student_id, details in students.items():
    print("ID:", student_id)

    for key, value in details.items():
        print(key, ":", value)
```

This technique is useful for API responses, JSON data, employee records, and configuration settings.

Avoid missing-key errors

If you are not sure whether a nested key exists, chained `get()` calls can help:

```
city = students.get("student3", {}).get("city", "Unknown")
print(city)
```

Output:

```
Unknown
```

This avoids `KeyError` if the outer key is missing.

---

11. Dictionary merging

Sometimes you need to combine two dictionaries.

Method 1: `update()`

```
a = {"x": 1, "y": 2}
b = {"y": 20, "z": 3}

a.update(b)

print(a)
```

Output:

```
{'x': 1, 'y': 20, 'z': 3}
```

`update()` modifies `a`. If keys overlap, values from `b` overwrite the corresponding values in `a`.

Method 2: Dictionary unpacking

```
a = {"x": 1, "y": 2}
b = {"y": 20, "z": 3}
```

```
combined = {**a, **b}

print(combined)
```

Output:

```
{'x': 1, 'y': 20, 'z': 3}
```

This creates a new dictionary.

Method 3: The `|` operator

In Python 3.9 and later:

```
a = {"x": 1, "y": 2}
b = {"y": 20, "z": 3}

combined = a | b

print(combined)
```

Output:

```
{'x': 1, 'y': 20, 'z': 3}
```

Method 4: `|=` to update in place

```
a = {"x": 1}
a |= {"y": 2}

print(a)
```

Output:

```
{'x': 1, 'y': 2}
```

Interview rule: When duplicate keys occur during a merge, the value from the right-hand dictionary wins.

12. Shallow copy vs deep copy

This is an important advanced interview concept.

Shallow copy

A shallow copy creates a new outer dictionary, but nested mutable objects remain shared.

```
original = {
    "name": "Aman",
    "skills": ["Python", "SQL"]
}

copied = original.copy()

copied["name"] = "Riya"
copied["skills"].append("Java")

print(original)
print(copied)
```

Output:

```
{'name': 'Aman', 'skills': ['Python', 'SQL', 'Java']}
{'name': 'Riya', 'skills': ['Python', 'SQL', 'Java']}
```

Why did the original skills change? Both dictionaries refer to the same nested list.

Deep copy

A deep copy recursively copies nested objects as well.

```
import copy

original = {
    "name": "Aman",
    "skills": ["Python", "SQL"]
}

copied = copy.deepcopy(original)
```

```
copied["skills"].append("Java")

print(original)
print(copied)
```

Output:

```
{'name': 'Aman', 'skills': ['Python', 'SQL']}
{'name': 'Aman', 'skills': ['Python', 'SQL', 'Java']}
```

Comparison

Shallow copy	Deep copy
Copies the outer container	Recursively copies nested objects
Nested mutable objects can be shared	Nested objects are generally independent
`d.copy()` or `dict(d)`	`copy.deepcopy(d)`

Deep copy has limitations for certain objects and may be more expensive in time and memory.

13. Dictionary keys and hashability

Dictionary keys must be **hashable**. This is why strings and integers are common keys.

Valid keys

```
data = {
    "name": "Aman",
    10: "ten",
    (1, 2): "tuple key"
}

print(data)
```

Invalid keys

```
data = {[1, 2]: "list key"}  # TypeError
```

Lists are mutable and unhashable, so they cannot be dictionary keys.

Can tuples be keys?

Yes, if all elements of the tuple are hashable.

```
data = {(1, 2): "valid"}
```

But this fails:

```
data = {(1, [2, 3]): "invalid"}  # TypeError
```

The tuple contains an unhashable list.

Duplicate keys

```
data = {
    "age": 20,
    "name": "Aman",
    "age": 25
}

print(data)
```

Output:

```
{'age': 25, 'name': 'Aman'}
```

The last value assigned to `"age"` wins. The key still has its original insertion position.

Important: `True` and `1`

In Python, `True == 1` and their hashes are compatible, so they cannot be distinct dictionary keys in the same dictionary.

```
data = {True: "yes", 1: "one"}

print(data)
```

The dictionary has one entry, whose value is `"one"`.

---

14. Dictionary coding interview questions with answers

These are common beginner-to-intermediate coding problems. Try to understand the logic and dry run, not just memorize the code.

Question 1: Count the frequency of each element

```
numbers = [1, 2, 2, 3, 1, 2, 4]

frequency = {}

for number in numbers:
    frequency[number] = frequency.get(number, 0) + 1

print(frequency)
```

Output:

```
{1: 2, 2: 3, 3: 1, 4: 1}
```

Dry run:

Number	Previous frequency	New frequency
1	0	1
2	0	1
2	1	2
3	0	1
1	1	2
2	2	3
4	0	1

The expression `frequency.get(number, 0) + 1` retrieves the current count, or starts at zero if the key is missing, and increments it.

Question 2: Count character frequency in a string

```
text = "banana"
frequency = {}

for char in text:
    frequency[char] = frequency.get(char, 0) + 1

print(frequency)
```

Output:

```
{'b': 1, 'a': 3, 'n': 2}
```

Question 3: Find the first non-repeating character

```
text = "aabbcde"
frequency = {}

for char in text:
    frequency[char] = frequency.get(char, 0) + 1

for char in text:
    if frequency[char] == 1:
        print(char)
        break
```

Output:

```
c
```

Logic: First count each character. Then scan the original string from left to right. The first character with frequency one is `c`.

Question 4: Find the most frequent element

```
numbers = [1, 2, 2, 3, 2, 4, 4]

frequency = {}
```

```
for number in numbers:
    frequency[number] = frequency.get(number, 0) + 1

most_frequent = max(frequency, key=frequency.get)

print(most_frequent)
```

Output:

```
2
```

`max(frequency, key=frequency.get)` finds the key with the largest associated value.

If multiple values tie for the maximum, this expression returns the first maximum key encountered in dictionary iteration order.

Question 5: Find the key with the maximum value

```
marks = {
    "Aman": 85,
    "Riya": 95,
    "Kabir": 78
}

top_student = max(marks, key=marks.get)

print(top_student)
```

Output:

```
Riya
```

Question 6: Filter dictionary entries by value

Find students whose marks are at least 80.

```
marks = {
    "Aman": 85,
    "Riya": 95,
```

```
    "Kabir": 78,
    "Sara": 70
}

passed = {
    name: score
    for name, score in marks.items()
    if score >= 80
}

print(passed)
```

Output:

```
{'Aman': 85, 'Riya': 95}
```

Question 7: Swap dictionary keys and values

```
original = {"a": 1, "b": 2, "c": 3}

result = {
    value: key
    for key, value in original.items()
}

print(result)
```

Output:

```
{1: 'a', 2: 'b', 3: 'c'}
```

This requires the original values to be hashable. If they are not unique, a value-to-key mapping will overwrite earlier entries.

Question 8: Merge two dictionaries

```
a = {"x": 10, "y": 20}
b = {"y": 30, "z": 40}

result = {**a, **b}
```

```
print(result)
```

Output:

```
{'x': 10, 'y': 30, 'z': 40}
```

The value `30` replaces `20` because `b` is unpacked after `a`.

Question 9: Group words by their first letter

```
words = ["apple", "ant", "banana", "ball", "cat"]
groups = {}

for word in words:
    first_letter = word[0]
    groups.setdefault(first_letter, []).append(word)

print(groups)
```

Output:

```
{
    'a': ['apple', 'ant'],
    'b': ['banana', 'ball'],
    'c': ['cat']
}
```

Dry run: `setdefault(first_letter, [])` creates an empty list when the letter is new. `.append(word)` adds the word to that letter's list.

This is a common interview pattern for grouping records.

Question 10: Find common keys in two dictionaries

```
a = {"x": 1, "y": 2, "z": 3}
b = {"y": 20, "z": 30, "w": 40}

common_keys = a.keys() & b.keys()
```

```
print(common_keys)
```

Output:

```
{'y', 'z'}
```

Question 11: Sum all dictionary values

```
marks = {"Python": 90, "SQL": 80, "Java": 85}

total = sum(marks.values())

print(total)
```

Output:

```
255
```

Question 12: Sort a dictionary by its values

```
marks = {
    "Aman": 85,
    "Riya": 95,
    "Kabir": 78
}

sorted_marks = dict(
    sorted(marks.items(), key=lambda item: item[1])
)

print(sorted_marks)
```

Output:

```
{'Kabir': 78, 'Aman': 85, 'Riya': 95}
```

Explanation:

- `items()` provides `(key, value)` pairs.
- `item[1]` selects the value.
- `sorted()` orders the pairs by value.
- `dict()` converts those pairs into a dictionary, preserving the sorted insertion order.

For descending order, use `reverse=True`:

```
sorted_marks = dict(
    sorted(marks.items(), key=lambda item: item[1], reverse=True)
)
```

Question 13: Remove duplicate values while keeping the first key

```
data = {"a": 10, "b": 20, "c": 10, "d": 30}

result = {}

for key, value in data.items():
    if value not in result.values():
        result[key] = value

print(result)
```

Output:

```
{'a': 10, 'b': 20, 'd': 30}
```

This is easy to understand, but checking `result.values()` on each iteration can make it slower for large dictionaries.

Question 14: Invert a dictionary when values repeat

Given:

```
data = {"Aman": "Python", "Riya": "SQL", "Kabir": "Python"}
```

Group the names by skill:

```
result = {}

for name, skill in data.items():
    result.setdefault(skill, []).append(name)

print(result)
```

Output:

```
{
    'Python': ['Aman', 'Kabir'],
    'SQL': ['Riya']
}
```

This is safer than a simple reversal when values repeat, because it keeps all associated keys.

Question 15: Build a dictionary from a list of words and their lengths

```
words = ["apple", "banana", "kiwi"]

result = {word: len(word) for word in words}

print(result)
```

Output:

```
{'apple': 5, 'banana': 6, 'kiwi': 4}
```

If a word appears more than once, its dictionary key will occur only once.

15. Time complexity of dictionaries

Let n be the number of entries in a dictionary.

Operation	Average time complexity
Access `d[key]`	$O(1)$
Insert or update `d[key] = value`	$O(1)$

Search for a key	$O(1)$
Delete a key	$O(1)$
Iterate over the dictionary	$O(n)$
Copy the dictionary	$O(n)$
Check whether a value exists in `d.values()`	$O(n)$

These are average-case expectations, not guarantees. Hash collisions and other implementation details can affect performance.

Interview answer: Python dictionaries use hash tables. Hashing helps Python locate the entry associated with a key efficiently.

16. Common mistakes to avoid

Mistake 1: Accessing a missing key directly

```
d = {"a": 10}
print(d["b"])  # KeyError
```

Use `d.get("b")` when the key may be missing.

Mistake 2: Thinking `get()` adds a key

```
d.get("b", 0)
```

This does not insert `"b"`. Use `setdefault()` or assignment if you want to add it.

Mistake 3: Assigning the result of `update()`

```
d = {"a": 1}
d = d.update({"b": 2})  # d is now None
```

Call `d.update({"b": 2})` without assigning its return value.

Mistake 4: Using mutable objects as keys

```
d = {[1, 2]: "value"}  # TypeError
```

Use an appropriate hashable key, such as `(1, 2)`.

Mistake 5: Confusing keys and values

```
d = {"age": 21}
print("age" in d)         # True
print(21 in d)            # False
print(21 in d.values())   # True
```

Mistake 6: Assuming a shallow copy is fully independent

A shallow copy creates a new outer dictionary but can share nested lists and dictionaries. Use `copy.deepcopy()` when independent nested copies are required.

17. Most-asked dictionary interview questions

Use these for quick revision.

1. **What is a dictionary?** A mutable, insertion-ordered mapping of unique keys to values.
2. **Are dictionary keys unique?** Yes. Assigning a value to an existing key replaces its previous value.
3. **Can dictionary values be duplicated?** Yes.
4. **Can a list be a dictionary key?** No, because a list is unhashable.
5. **What is the difference between `get()` and square brackets?** `get()` handles missing keys with a default; square brackets raise `KeyError`.
6. **What is the difference between `get()` and `setdefault()`?** `get()` reads; `setdefault()` also inserts a default for a missing key.
7. **What does `items()` return?** A dynamic view of key-value pairs.
8. **What does `pop()` return?** The value associated with the removed key.
9. **What does `popitem()` remove?** The last inserted key-value pair.
10. **What does `update()` return?** `None`; it modifies the dictionary.
11. **What is dictionary comprehension?** A compact way to create dictionaries from iterables.
12. **What is a nested dictionary?** A dictionary containing other dictionaries as values.
13. **What is the difference between shallow and deep copy?** Shallow copy may share nested mutable objects; deep copy recursively copies them.
14. **How do you merge dictionaries?** With `update()`, unpacking, or the `|` operator in Python 3.9+.
15. **What is the average lookup complexity?** $O(1)$.

16. **How do you count element frequencies?** Use a dictionary to track each element's count.
17. **How do you sort a dictionary by value?** Sort its `items()` using the value as the sorting key.
18. **Does a dictionary preserve insertion order?** Yes, as a language guarantee in Python 3.7 and later.
19. **Can tuples be dictionary keys?** Yes, if all their elements are hashable.
20. **Can a dictionary contain another dictionary as a value?** Yes, which is useful for structured data.

18. Test your knowledge

Dictionary quiz

0/7 answered

1. What does `d.get('x', 0)` do when `x` is missing?

- Adds x with value 0
- Returns 0 without adding x
- Raises KeyError

2. Which method removes a key and returns its value?

- clear()
- pop()
- update()

3. What does `d.items()` provide?

- Only keys
- Only values
- Key-value pairs

4. What happens when you assign a value to an existing key?

- A duplicate key is created
- The old value is replaced

- The dictionary is cleared

5. Which statement about dictionary keys is correct?

- Lists are valid keys
- Dictionaries are valid keys
- Keys must be hashable

6. What is the average lookup complexity?

- O(1)
- O(n)
- $O(n^2)$

7. Which method groups values into a list when used with `.append()`?

- setdefault()
- clear()
- popitem()

Reset quiz

19. Final dictionary mastery checklist

Your learning progress 0%

- Create dictionaries using {}, dict(), and fromkeys().
- Access values with [] and get().
- Add and update entries using assignment and update().
- Understand get(), setdefault(), pop(), and popitem().
- Use keys(), values(), and items().
- Delete entries with del, pop(), and clear().

- Loop through dictionaries and check key membership.
- Write dictionary comprehensions.
- Access and update nested dictionaries.
- Merge dictionaries and handle duplicate keys.
- Understand hashable keys, shallow copy, and deep copy.
- Count frequencies and find the first non-repeating character.
- Group data and sort dictionaries by value.
- Explain dictionary time complexity and common interview pitfalls.

Recommended study order: First master dictionary creation, access, update, and deletion. Next practice methods and loops. Then move to dictionary comprehension, nested dictionaries, copying, and frequency-counting problems. Finally, solve the interview questions without looking at the answers.